Terraform Provider

> **Series:** AUTOM | **Notebook:** 4 of 8 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Terraform CLI	Version 1.0+ installed
API Token	Token with `settings.read`, `settings.write`, `ReadConfig`, `WriteConfig`
Tenant URL	Your Dynatrace SaaS tenant URL
HCL Knowledge	Basic familiarity with Terraform syntax

Learning Objectives

By the end of this notebook, you will:

- Understand the Dynatrace Terraform provider
- Know how to configure resources in HCL
- Be able to manage state and handle drift
- Implement multi-environment deployments

---

1. Introduction

The Dynatrace Terraform provider enables infrastructure-as-code management of Dynatrace configurations. It integrates with Terraform's ecosystem for state management, planning, and CI/CD integration.

Why Terraform?

Benefit	Description
State Management	Track what exists vs. what's defined
Drift Detection	Identify manual changes
Planning	Preview changes before applying
Ecosystem	Integrate with other Terraform providers
Modules	Reusable configuration packages

Terraform vs Monaco

Aspect	Terraform	Monaco
State file	Required	None
Drift detection	Built-in	Manual
Learning curve	Higher	Lower
Multi-cloud	Yes	Dynatrace only
Dependencies	Explicit	Implicit

---

2. Getting Started

Installation

Install Terraform:

macOS:

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux:

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update &amp;amp;&amp;amp; sudo apt install terraform
```

Verify installation:

```
terraform version
```

---

3. Provider Configuration

Basic Setup

Create a `main.tf` file:

```
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~&gt; 1.0"
    }
  }
}

provider "dynatrace" {
  dt_env_url   = var.dynatrace_url
  dt_api_token = var.dynatrace_token
}
```

Variables File

Create a `variables.tf` file:

```
variable "dynatrace_url" {
  description = "Dynatrace environment URL"
  type        = string
}

variable "dynatrace_token" {
  description = "Dynatrace API token"
  type        = string
  sensitive   = true
}
```

Environment-Specific Values

Create `terraform.tfvars`:

```
dynatrace_url   = "https://abc12345.live.dynatrace.com"
dynatrace_token = "&lt;your-api-token&gt;"
```

> **Security:** Never commit `terraform.tfvars` with real tokens. Use environment variables or secret management.

Initialize and Validate

```
# Initialize provider
terraform init

# Validate configuration
terraform validate

# Format code
terraform fmt
```

---

4. Resource Types

Management Zone

```
resource "dynatrace_management_zone_v2" "production" {
  name = "Production"

  rules {
    type    = "SERVICE"
    enabled = true

    conditions {
      condition {
        key {
          type      = "STATIC"
          attribute = "SERVICE_TAGS"
        }
        tag {
          operator = "EQUALS"
          negate   = false
          value {
            context = "CONTEXTLESS"
            key     = "environment"
            value   = "production"
          }
        }
      }
    }
  }
}
```

Auto-Tagging Rule

```
resource "dynatrace_autotag_v2" "application" {
  name = "Application"

  rules {
    type         = "SERVICE"
    enabled      = true
    value_format = "{Service:DetectedName}"
    conditions   = []
  }
}
```

Alerting Profile

```
resource "dynatrace_alerting" "production_alerts" {
  name            = "Production Alerting"
  management_zone = dynatrace_management_zone_v2.production.id

  rules {
    delay_in_minutes = 0
    severity_level   = "AVAILABILITY"
    tag_filters      = []
  }

  rules {
    delay_in_minutes = 5
    severity_level   = "ERRORS"
    tag_filters      = []
  }

  rules {
    delay_in_minutes = 10
    severity_level   = "PERFORMANCE"
    tag_filters      = []
  }
}
```

SLO

```
resource "dynatrace_slo_v2" "availability" {
  name              = "Production Availability"
  enabled           = true
  evaluation_type   = "AGGREGATE"
  evaluation_window = "-1w"
  target_success    = 99.9
  target_warning    = 99.95

  metric_expression =
"builtin:synthetic.http.availability.location.total:splitBy()"

  filter            = "type(SYNTHETIC_TEST)"
}
```

HTTP Monitor (Synthetic)

```
resource "dynatrace_http_monitor" "homepage" {
  name      = "Homepage Check"
  enabled   = true
  frequency = 5

  locations = ["GEOLOCATION-1234567890ABCDEF"]

  anomaly_detection {
    loading_time_thresholds {
      enabled = true
    }
    outage_handling {
      global_outage = true
      local_outage  = false
    }
  }

  script {
    request {
      description = "Homepage"
      method      = "GET"
      url         = "https://example.com"

      validation {
        rule {
          type  = "httpStatusesList"
          value = "&gt;=400"
          pass_if_found = false
        }
      }
    }
  }
}
```

---

5. State Management

Understanding Terraform State

Terraform tracks resources in a state file (`terraform.tfstate`):

Concept	Description
State	JSON file mapping config to real resources
Plan	Comparison of state vs. desired config
Apply	Execute changes to reach desired state
Drift	Difference between state and actual

Remote State Storage

For team collaboration, use remote state:

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "dynatrace/production/terraform.tfstate"
    region = "us-east-1"
  }
}
```

Or using Terraform Cloud:

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "dynatrace-production"
    }
  }
}
```

Common State Commands

```
# Preview changes
terraform plan

# Apply changes
terraform apply

# Show current state
terraform show

# List resources in state
terraform state list

# Import existing resource
terraform import dynatrace_management_zone_v2.production "&lt;object-id&gt;"

# Remove resource from state (without deleting)
terraform state rm dynatrace_management_zone_v2.production
```

Importing Existing Resources

To manage existing configurations:

1. Write the resource block in HCL
2. Find the object ID in Dynatrace
3. Import into state

```
terraform import dynatrace_management_zone_v2.production
"vu9U3hXa3q0AAAABABhidWlsdGluOm1hbmFnZW1lbnQtem9uZXMABnRlbmFudAAGdGVuYW50ABp2
dTlVM2hYYTNxMERUVF9fUHJvZHVjdGlvbr7vFJ4"
```

---

6. Advanced Patterns

Modules for Reusability

Create reusable modules:

modules/environment/main.tf:

```
variable "environment_name" {
  type = string
}

variable "environment_tag" {
  type = string
}

resource "dynatrace_management_zone_v2" "zone" {
  name = var.environment_name

  rules {
    type    = "SERVICE"
    enabled = true

    conditions {
      condition {
        key {
          type      = "STATIC"
          attribute = "SERVICE_TAGS"
        }
        tag {
          operator = "EQUALS"
          negate   = false
          value {
            context = "CONTEXTLESS"
            key     = "environment"
            value   = var.environment_tag
          }
        }
      }
    }
  }
}

output "management_zone_id" {
  value = dynatrace_management_zone_v2.zone.id
}
```

Use the module:

```
module "production" {
  source           = "./modules/environment"
  environment_name = "Production"
  environment_tag  = "production"
}

module "staging" {
  source           = "./modules/environment"
  environment_name = "Staging"
  environment_tag  = "staging"
}
```

Workspaces for Environments

Use workspaces to manage multiple environments:

```
# Create workspaces
terraform workspace new development
terraform workspace new staging
terraform workspace new production

# Switch workspace
terraform workspace select production

# List workspaces
terraform workspace list
```

Use workspace in config:

```
locals {
  environment = terraform.workspace

  config = {
    development = {
      alert_delay = 30
    }
    staging = {
      alert_delay = 15
    }
    production = {
      alert_delay = 5
    }
  }
}

resource "dynatrace_alerting" "alerts" {
  name = "${local.environment} Alerting"

  rules {
    delay_in_minutes = local.config[local.environment].alert_delay
    severity_level   = "AVAILABILITY"
  }
}
```

Best Practices

Practice	Description
Remote state	Never use local state for teams
State locking	Enable to prevent concurrent changes
Modular design	Use modules for reusable patterns
Version pinning	Pin provider versions
Plan before apply	Always review plan output
Meaningful names	Resource names should be descriptive

Common Issues

Issue	Cause	Solution
"Resource already exists"	State out of sync	Import the resource
"Provider error"	Invalid token	Check credentials
"Drift detected"	Manual changes	Re-apply or update state
Slow plan/apply	Many resources	Use targets or modules

---

7. Next Steps

Terraform Export Utility

To export existing Dynatrace configuration to Terraform:

```
# Install terraform-provider-dynatrace-export
# See: https://github.com/dynatrace-oss/terraform-provider-dynatrace

# Export all configurations
terraform-provider-dynatrace-export \
  -e https://{tenant}.live.dynatrace.com \
  -t {api-token} \
  -o ./exported-config
```

Continue the Series

Next Notebook	Focus
AUTOM-05: Dynatrace Workflows	Event-driven automation

Additional Resources

- Terraform Provider Documentation
- Provider GitHub Repository
- Terraform Style Guide

Summary

In this notebook, you learned:

- How to configure the Dynatrace Terraform provider
- Creating resources like management zones, auto-tags, and SLOs
- State management and drift detection
- Advanced patterns with modules and workspaces

> **Key Takeaway:** Terraform excels when you need state management, drift detection, or integration with other infrastructure. For Dynatrace-only config management, Monaco is often simpler.

Continue to AUTOM-05: Dynatrace Workflows to learn event-driven automation.
